

SQL Day 1 - Revision Report

Purpose: 3-day active-recall revision sheet for Data Engineer SQL preparation.

1. Conditional Aggregation

Use conditional aggregation when several metrics are required for the same group, but only selected rows should contribute.

```
SUM(
CASE
WHEN condition THEN value
ELSE 0
END
)
```

Conditional count: THEN 1. Conditional amount: THEN Amount. Non-matching rows contribute 0. Mental rule: ask, "What should this row contribute to the final SUM?"

2. Conditional Aggregation + HAVING

HAVING filters grouped/aggregated results. WHERE filters individual rows before grouping.

```
HAVING SUM(
CASE WHEN Status = 'Completed'
THEN 1 ELSE 0 END
) >= 2
```

Core execution intuition: WHERE -> GROUP BY -> HAVING.

3. LEFT JOIN + Aggregation

LEFT JOIN preserves every row from the left table, including customers with no matching orders.

```
SELECT C.Customer_ID,
COUNT(O.Order_ID) AS Total_Orders
FROM Customers C
LEFT JOIN Orders O
ON C.Customer_ID = O.Customer_ID
GROUP BY C.Customer_ID;
```

After a LEFT JOIN, COUNT(O.Order_ID) counts real matching orders and returns 0 for customers with none. COUNT(*) may count the preserved left-side row.

4. Safe Division with NULLIF

Use NULLIF to protect calculations where the denominator may be zero.

```
100.0 * Completed_Orders
/ NULLIF(Total_Orders, 0)
```

Mental rule: possible denominator = 0 -> think NULLIF.

5. String Functions

Functions practiced for cleaning and transformation:

```
LOWER(TRIM(Full_Name))
SPLIT_PART(Email, '@', 2)
REPLACE(Phone, '-', '')
SUBSTRING('DATABASE2026', 5, 4)
LENGTH(Product_Code)
```

TRIM removes surrounding spaces. LOWER/UPPER normalize case. SPLIT_PART extracts a delimited section. REPLACE substitutes text. SUBSTRING extracts by start position and length. LENGTH counts characters.

6. String Cleaning + CASE

Combine transformations with CASE to validate incoming data.

```

SELECT
Customer_ID,
LOWER(TRIM(Full_Name)) AS Clean_Name,
SPLIT_PART(Email, '@', 2) AS Email_Domain,
REPLACE(Phone, '-', '') AS Clean_Phone,
CASE
WHEN LENGTH(REPLACE(Phone, '-', '')) = 10
THEN 'Valid'
ELSE 'Invalid'
END AS Phone_Status
FROM Customers;

```

Data Engineering pattern: clean -> transform -> validate -> classify.

7. Date Extraction

EXTRACT retrieves components such as YEAR, MONTH, or DAY from a date.

```

EXTRACT(YEAR FROM Order_Date)
EXTRACT(MONTH FROM Order_Date)
EXTRACT(DAY FROM Order_Date)

```

Monthly analytics pattern: extract date component -> group by it -> aggregate.

8. Monthly / Year-Month Aggregation

Group by the extracted date components when metrics are required per period.

```

SELECT
EXTRACT(YEAR FROM Order_Date) AS Year,
EXTRACT(MONTH FROM Order_Date) AS Month,
SUM(Amount) AS Total_Sales
FROM Orders
GROUP BY
EXTRACT(YEAR FROM Order_Date),
EXTRACT(MONTH FROM Order_Date);

```

For interview-safe SQL, repeat expressions in GROUP BY rather than relying on SELECT aliases when the SQL dialect is unspecified.

9. SQL Logical Processing Reminder

Useful conceptual order for understanding WHERE, GROUP BY, HAVING, and aliases:

```

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY

```

SELECT aliases are created after GROUP BY conceptually. ORDER BY generally can use SELECT aliases because it occurs later.

10. Filtering Dates

Filtering individual orders by year/month is a row-level operation, so use WHERE.

```

SELECT Order_ID
FROM Orders
WHERE EXTRACT(YEAR FROM Order_Date) = 2026
AND EXTRACT(MONTH FROM Order_Date) = 3;

```

EXTRACT(MONTH ...) normally returns a number such as 3, not the string 'MARCH'.

11. DATEDIFF

Assuming the dialect DATEDIFF(day, start_date, end_date), calculate elapsed days and filter individual rows with WHERE.

```

SELECT
Order_ID,
DATEDIFF(day, Order_Date, Delivery_Date) AS Delivery_Days

```

```
FROM Orders
WHERE DATEDIFF(day, Order_Date, Delivery_Date) > 5;
```

Date-function syntax varies across SQL dialects, so state the assumed dialect in an interview when necessary.

12. DATEADD

Assuming DATEADD(day, number, date), add or subtract days from a date.

```
SELECT
Order_ID,
Order_Date,
DATEADD(day, 10, Order_Date) AS Payment_Due_Date
FROM Orders;
```

Positive number moves forward; a negative number moves backward.

13. Final Mixed Pattern

This query combines date extraction, grouping, total count, conditional count, and conditional sum.

```
SELECT
EXTRACT(MONTH FROM O.Order_Date) AS Month,
COUNT(O.Order_ID) AS Total_Orders,
SUM(CASE
WHEN O.Status = 'Completed' THEN 1
ELSE 0
END) AS Completed_Orders,
SUM(CASE
WHEN O.Status = 'Completed' THEN O.Amount
ELSE 0
END) AS Completed_Amount
FROM Orders O
GROUP BY EXTRACT(MONTH FROM O.Order_Date);
```

This is the key analytical composition from the session: multiple metrics at one grouping level in a single query.

3-Day Active Recall Checklist

1. Explain WHERE vs HAVING without looking at notes.
2. Write a conditional count using SUM(CASE WHEN ...).
3. Write a conditional amount using SUM(CASE WHEN ...).
4. Explain COUNT(*) vs COUNT(column) after a LEFT JOIN.
5. Clean a name using TRIM + LOWER.
6. Extract a domain from an email.
7. Remove hyphens from a phone number.
8. Validate a cleaned phone number using LENGTH + CASE.
9. Explain SUBSTRING(text, start, length).
10. Extract month and year from a date.
11. Calculate monthly sales.
12. Calculate days between two dates.
13. Add 10 days to a date.
14. Write one query combining date grouping + conditional aggregation.

Revision strategy: Attempt these from memory before reading the examples. The goal is retrieval and reasoning, not rereading.